

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
CO3 ASSESSMENT TOOL 2 – Code Review & Repository Evaluation

Course Code:	CSA1003	Course Title:	Software Engineering
CO Assessed:	CO3	Assessment:	Assessment Tool 1 – Code Review & Repository Evaluation
Weightage:	20%	Total Marks:	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:	ALIGETI AKSHAYA	Reg. No.:	192425170
Date:	17-08-2026	Duration:	60 minutes

Code of Conduct

I A.Akshaya(192425170) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Akshaya

Annexure A – Code Review & Repository Evaluation

Instructions to Students:

**ACADEMIC PROJECT, ALLOCATION MONITORING AND EVALUATION
MANAGEMENT SYSTEM**

Assignment Questions

1. Problem Overview

- Explain the assigned problem statement and summarize the implemented solution.

The **Academic Project, Allocation Monitoring and Evaluation Management System** is developed to solve the problem of assigning academic projects without properly considering students' individual interests and technical skills. The system provides a centralized platform where students can enter their interests and skills, and suitable projects can be allocated based on this information. Faculty members can guide students and monitor their progress, while project coordinators, HODs, and management can track project activities and evaluation details.

- Describe the project objectives and key functionalities.

Project Objectives

- Allocate projects according to students' interests and technical skills.
- Make project allocation fair and transparent.
- Help students improve their practical knowledge.
- Allow faculty members to monitor project progress.
- Maintain project details and evaluation records in one place.
- Improve communication between students, faculty, and management.
- Reduce manual work involved in project allocation and monitoring.

Key Functionalities

- User Registration and Login
- Student Profile Management
- Interest and Skill Selection
- Project Allocation
- Faculty Guide Assignment
- Project Progress Tracking
- Evaluation and Feedback
- Deadline Management
- Student and Faculty Dashboards
- Report Generation
- Role-Based Access Control

2. Repository Organization

- Evaluate the repository structure, folder organization, and file naming conventions.

academic-project-management/

|

|— frontend/

| |— public/

| |— src/

| |— components/

| |— pages/

| └─ services/

| └─ assets/

| └─ styles/

| └─ App.js

|

└─ backend/

| └─ controllers/

| └─ models/

| └─ routes/

| └─ middleware/

| └─ services/

| └─ config/

| └─ server.js

|

└─ database/

| └─ schema.sql

| └─ seed.sql

|

└─ docs/

| └─ requirements/

| └─ diagrams/

| └─ reports/

|

└─ tests/

|

```
|— .gitignore
|— README.md
└— package.json
```

Folder Organization

- **frontend/** contains the user interface and screens.
- **backend/** contains server-side logic and APIs.
- **database/** contains database tables and sample data.
- **docs/** contains SRS, UML diagrams, architecture diagrams, and reports.
- **tests/** contains test cases and testing files.

File Naming Conventions

Meaningful names are used for files so that their purpose can be understood easily.

For example:

```
StudentDashboard.js
ProjectAllocation.js
Login.js
projectController.js
studentModel.js
authMiddleware.js
```

- Justify how the repository layout supports maintainability and collaboration.

This organization supports **maintainability** because developers can easily locate and modify a particular module. It also supports **collaboration** because different team members can work on different folders without disturbing other parts of the project.

3. Code Quality Review

- Review the source code for readability, modularity, coding standards, and documentation.

Readability

- Use meaningful variable and function names.
- Maintain proper indentation.
- Keep functions small and understandable.
- Avoid unnecessary or repeated code.
- Use comments where the logic needs additional explanation.

For example:

```
function allocateProject(student, project) {  
  
    // Allocate a project based on student's skills  
  
}
```

The function name clearly explains its purpose.

Modularity

The application is divided into separate modules such as:

- Login and authentication
- Student profile
- Project allocation
- Faculty monitoring
- Evaluation
- Reports

This makes it easier to modify one feature without affecting the complete application.

Coding Standards

The project should follow:

- Consistent naming conventions.
- Proper indentation.
- Meaningful function names.
- Proper error handling.
- Reusable components.
- Avoidance of duplicate code.

Documentation

Important parts of the project should be documented using:

- README.md
- Code comments
- API documentation
- SRS document
- UML diagrams
- Architecture diagrams
 - Identify strengths and areas for improvement.

Strengths

The major strengths of the code structure are:

- **Modular:** Features are separated into different components.
- **Readable:** Meaningful names make the code easier to understand.
- **Maintainable:** Changes can be made to individual modules.
- **Reusable:** Common components can be reused.
- **Scalable:** New modules can be added as the system grows.

Areas for Improvement

Some improvements can be made as the project develops:

- Add more automated unit and integration tests.
- Improve error handling.
- Use a common coding style across all modules.
- Add stronger input validation.
- Improve security for authentication and user data.
- Remove unused code and dependencies.
- Add proper API documentation.
- Use code review before merging major changes.

4. Version Control Evaluation

- Analyze the Git repository by reviewing commit history, branching strategy, commit messages, and repository maintenance practices.

Commit History

Meaningful commits are used to record major changes in the project.

Initial project structure

Add user registration and login

Add student profile module

Add interest and skill selection

Implement project allocation

Add faculty dashboard

Add project progress tracking

Add evaluation and feedback

Fix login validation

Add report generation

Prepare version 1.0 release

Branching Strategy

The project uses:

- **main** – Stable and tested version.
- **development** – Latest combined development work.
- **feature/*** – Used for individual features.
- **release/*** – Used for final testing before release.
- **hotfix/*** – Used to quickly fix important problems.

For example:

main

|

└─ development

└─ feature/login

└─ feature/project-allocation

└─ feature/evaluation

└─ feature/dashboard

Repository Maintenance

The repository can be maintained by:

- Regularly committing completed work.
 - Removing unnecessary files.
 - Keeping the .gitignore file updated.
 - Using meaningful commit messages.
 - Reviewing code before merging.
 - Keeping the README.md updated.
 - Avoiding passwords and sensitive information in the repository.
- Explain how version control supports collaborative software development.

Git allows multiple developers to work on different features at the same time. For example, one developer can work on the **login module**, while another works on **project allocation**. Feature branches prevent unfinished code from affecting the stable version.

Git also helps developers:

- Track changes.
- Compare different versions.
- Resolve conflicts.
- Recover previous versions.
- Merge completed features.
- Work together using a remote repository such as GitHub.

5. Code Testing and Validation

- Demonstrate that the application functions correctly through appropriate testing.

Test ID	Module	Test Case	Expected Result	Status
T01	Registration	Enter valid student	Student account	Pass

T e s t I D	Modul e	Test Case	Expect ed Result	St at us
		detail s	t is created	
T C 0 2	Registr ation	Leav e requi red field empt y	Validat ion messag e is display ed	Pa ss
T C 0 3	Login	Enter valid email and pass word	User succes sfully logs in	Pa ss
T C 0 4	Login	Enter incor rect pass word	Error messag e is display ed	Pa ss
T C 0 5	Studen t Profile	Enter stude nt intere sts and skills	Profile is update d	Pa ss
T C 0 6	Project Allocat ion	Selec t suita ble proje ct	Project is allocat ed to student	Pa ss
T C 0 7	Facult y Dashb oard	Facul ty opens assign ed proje cts	Assign ed student s are display ed	Pa ss

T e s t I D	Modul e	Test Case	Expect ed Result	St at us
T C 0 8	Progre ss Tracki ng	Stude nt subm its progr ess repor t	Report is stored succes sfully	Pa ss
T C 0 9	Evalua tion	Facul ty enter s evalu ation mark s	Marks are saved	Pa ss
T C 1 0	Feedba ck	Facul ty subm its feedb ack	Feedba ck is display ed to student	Pa ss
T C 1 1	Role Access	Stude nt tries to acces s Admi n page	Access is denied	Pa ss
T C 1 2	Report s	Gene rate proje ct repor t	Report is generat ed succes sfully	Pa ss

- Present test cases, execution results, and screenshots as evidence.

1. Registration Testing

Enter valid student information and verify that the account is created.

2. Login Testing

Test both valid and invalid login details.

3. Profile Testing

Check whether students can enter and update their interests, programming languages, and technical skills.

4. Project Allocation Testing

Verify whether the project coordinator can assign projects based on student interests and skills.

5. Monitoring Testing

Check whether faculty members can view assigned students and their project progress.

6. Evaluation Testing

Verify that evaluation marks and feedback are stored correctly.

7. Security Testing

Check whether users can access only the features allowed for their role.

Evidence of Testing

Screenshots can be included in the report as evidence of successful execution.

You can take screenshots of:

1. **Login Page** – Shows successful user login.
2. **Student Profile** – Shows student interests and technical skills.

3. **Project Allocation Page** – Shows an allocated project.
4. **Student Dashboard** – Shows project details and progress.
5. **Faculty Dashboard** – Shows assigned students and project status.
6. **Evaluation Page** – Shows marks and feedback.
7. **Database** – Shows that submitted information is stored.
8. **Git Repository** – Shows commits and branches.
9. **Docker Desktop/Terminal** – Shows the application containers running successfully.

Testing Result

After testing the major modules, the system should successfully perform **registration, login, profile management, interest selection, project allocation, project monitoring, evaluation, feedback, and report generation**. Any errors identified during testing can be corrected and committed to Git with a meaningful commit message.

6. Repository Documentation

- Evaluate the completeness of the project documentation, including the README, installation instructions, usage guide, and dependencies.

README

The README.md file provides an overview of the project. It should explain:

- Project title and purpose.
- Problem being solved.
- Main objectives.
- Technologies used.
- Major features.
- Project folder structure.
- How to install and run the application.

Installation Instructions

The repository should clearly explain the software and tools required before running the project.

For example:

Node.js

MySQL

Git

Docker

Web Browser

The README can provide commands such as:

```
git clone <repository-url>
```

```
cd academic-project-management
```

```
npm install
```

If Docker is used:

```
docker compose up --build
```

- Suggest improvements to enhance usability and maintainability.

Usage Guide

The documentation should explain how each type of user interacts with the system.

For example:

- **Student** → Login → Update profile → Select interests and skills → View allocated project → Submit progress.
- **Faculty** → Login → View assigned students → Monitor progress → Provide feedback → Enter evaluation marks.
- **Project Coordinator** → View student skills → Allocate projects → Set deadlines → Monitor project status.
- **HOD/Management** → View project progress → Check evaluation details → Generate reports.
- **Admin** → Manage users and system information.

Documentation Structure

A better README.md can follow this structure:

Academic Project Management System

1. Project Overview
2. Problem Statement
3. Objectives
4. Features
5. Technologies Used
6. System Requirements
7. Folder Structure
8. Installation Instructions
9. Database Setup
10. Running the Application
11. User Guide
12. API Documentation
13. Docker Deployment
14. Git Workflow
15. Testing
16. Screenshots
17. Future Enhancements
18. Contributors

7. Code Review Findings and Recommendations

- Summarize the overall code review findings.
 - The project is divided into separate **frontend, backend, database, and testing** sections.
 - Meaningful names are used for folders, files, functions, and variables.
 - The application features are divided into separate modules such as **login, student profile, project allocation, monitoring, and evaluation.**

- Git branches can be used to develop different features independently.
- Meaningful commit messages help track project changes.
- Basic testing is performed for important functionalities.
- Documentation provides information about installation, usage, and project structure.
- Input validation and role-based access should be carefully maintained to improve security.
 - Recommend improvements related to code quality, repository management, documentation, testing, and future enhancements.

Recommendations for Code Quality

- Follow consistent coding standards throughout the project.
- Use meaningful variable and function names.
- Avoid duplicate code by creating reusable components.
- Keep functions small and focused on one task.
- Add proper error handling.
- Add comments only where they are useful for understanding complex logic.
- Remove unused code and unnecessary dependencies.

Recommendations for Repository Management

- Use separate branches for each major feature.
- Use meaningful commit messages such as:

Add student profile module

Implement project allocation

Fix login validation

Add faculty dashboard

- Review code before merging branches.
- Keep the main branch stable.
- Update .gitignore regularly.
- Never upload passwords, API keys, or other sensitive information.

Recommendations for Documentation

The README.md should be kept updated with:

- Project overview.
- Installation steps.
- Technologies used.
- Folder structure.
- Database setup.
- Usage instructions.
- API details.
- Docker deployment steps.
- Testing procedure.
- Screenshots.
- Future enhancements.

Recommendations for Testing

Testing should be expanded to include:

- **Unit testing** for individual functions.
- **Integration testing** for frontend-backend communication.
- **Database testing** for storing and retrieving information.
- **Authentication testing** for different user roles.
- **Security testing** for unauthorized access.
- **Performance testing** when the number of users increases.

Expected Deliverables

- Code Review Report (10–15 pages)
- Git repository link
- Screenshots of repository structure and commit history
- Code review observations with examples
- Test execution screenshots
- Recommendations for improvement

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Analysis & Repository Organization(15 %)	Clearly explains the problem, solution, and repository structure with logical organization and justification.	Good explanation with minor omissions.	Basic explanation and repository organization.	Incomplete or poorly organized repository.
Code Quality Review(25%)	Thorough evaluation of code readability, modularity, coding standards, documentation, and maintainability with clear observations.	Good code review with minor gaps.	Basic review with limited analysis.	Superficial or inaccurate code review.
Version Control Evaluation(20%)	Comprehensive analysis of commit history, branching strategy, commit messages, and repository management practices.	Good evaluation with adequate explanation.	Basic evaluation with limited evidence.	Poor or incomplete version control analysis.
Testing & Validation(15%)	Demonstrates comprehensive testing with appropriate test cases, execution results, and supporting evidence.	Good testing with minor omissions.	Basic testing with limited evidence.	Inadequate or missing testing and validation.
Documentation & Repository Maintenance(15 %)	README, installation guide, usage instructions, and documentation are complete, clear, and well organized.	Documentation is mostly complete.	Basic documentation with missing details.	Poor or insufficient documentation.
Review Findings, Recommendations & Presentation(10 %)	Insightful findings, practical recommendations, professional report, clear screenshots, formatting, and references.	Good recommendations and presentation.	Basic recommendations with acceptable presentation.	Weak conclusions and poor presentation.

Criteria	Mark
System Requirement Analysis (30%)	/5
Selection of Software Architecture (25%)	/5
Architecture Diagram and Component Design (20%)	/5
Component Interaction and Quality Attribute Analysis (15%)	/5
Architectural Justification and Technical Presentation (10%)	/5
TOTAL	/25

